

GKE 上的程式碼生成代理程式

來源：<https://codelabs.developers.google.com/next26/code-generation-agent-gke?hl=zh-tw>

步驟數：10

瞭解如何使用 ADK，在 Google Kubernetes Engine (GKE) 建構及部署安全的程式碼生成代理。您將設定混合模型策略，從本機 Qwen 模型回溯至 Vertex AI Gemini、在 Agent Sandbox (gVisor) 中安全地執行生成的程式碼，並透過 GKE Inference Gateway 和 Dynamic Resource Allocation (DRA) 最佳化推論服務。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [佈建基礎架構和動態資源分配 \(DRA\)](#)
4. [使用 llm-d 和 DRA 部署智慧型推論排程](#)
5. [使用 GKE Inference Gateway 設定智慧型路徑](#)
6. [使用 Agent Sandbox 安全執行程式碼](#)
7. [建構及部署程式碼生成代理 \(ADK\)](#)
8. [觀察及驗證](#)
9. [清除所用資源](#)
10. [摘要](#)

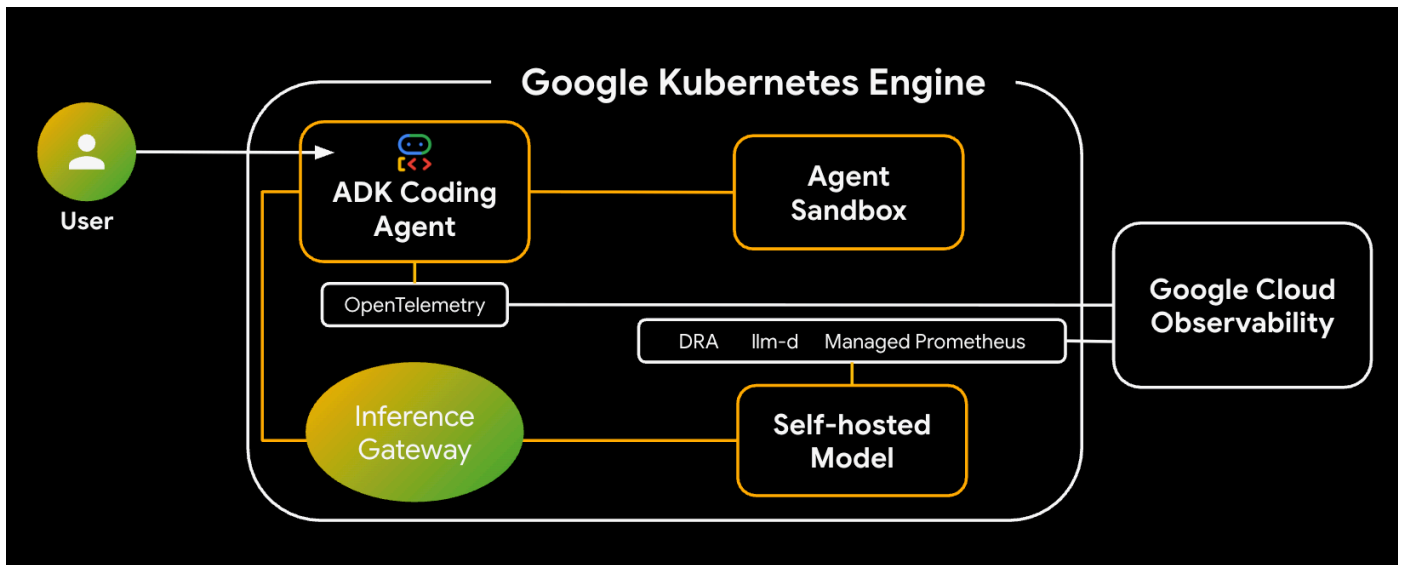
1. 簡介

總覽

在本實驗室中，您將瞭解如何在 Google Kubernetes Engine (GKE) 上建構及部署安全程式碼生成代理程式。程式碼生成代理程式需要執行可能不受信任的程式碼，因此需要安全的沙箱環境。您也會瞭解如何設定採用混合模型策略的代理程式，以便從 GKE 上的自架開放模型回復至 Vertex AI 的代管 Gemini 服務，進而提高可靠性。此外，您也會瞭解如何使用 GKE Inference Gateway 和動態資源分配 (DRA) 功能，最佳化推論服務。最後，您將瞭解如何運用 Google Cloud Observability，透過 Managed Prometheus 監控推論堆疊。

架構

您要建構的系統架構如下：



主要元件與優點

- **動態資源分配 (DRA)**：本實驗室使用這項功能，為模型伺服器 Pod 動態宣告及分配特定 GPU 資源 (NVIDIA L4)，確保推論工作負載能精確鎖定硬體。瞭解 [GKE 的 DRA](#)。
- **llm-d 和 vLLM**：提供模型服務框架和 Helm 資訊套件，用於部署 Qwen 模型。在本實驗室中，它會處理推論要求，並與 DRA 整合以進行資源管理 (本實驗室未啟用分離式服務)。請參閱 [llm-d 指南](#)，並查看 [llm-d GitHub 存放區](#)。
- **GKE Inference Gateway**：直接將 AI 推論感知路由邏輯移至負載平衡器。在本實驗室中，這項服務會將要求路徑最佳化，盡量提高前置字元快取命中率，進而縮短第一個權杖生成時間 (TTFT) 的延遲。瞭解 [推論閘道概念](#)。
- **代理程式沙箱 (gVisor)**：為執行 AI 代理程式產生的程式碼提供安全隔離機制。這項功能會使用 gVisor 提供深層核心隔離，保護主機節點免受不受信任的工作負載影響。瞭解 [GKE 上的 Agent Sandbox](#) 和 [GKE Sandbox Pods](#)。

學習內容

- **佈建基礎架構**：設定 GKE 叢集，並使用動態資源分配 (DRA) 管理 GPU。
- **部署推論堆疊**：部署 llm-d 和 vLLM，並採用智慧型推論排程。
- **設定智慧型路由**：使用 GKE Inference Gateway 進行前置字元快取感知路由。
- **安全執行程式碼**：部署 Agent Sandbox (gVisor)，安全執行 AI 生成的程式碼。
- **觀察及驗證**：使用 Google Cloud Monitoring 和 Managed Prometheus 查看提供模型指標。

課程內容

- 如何在 GKE 中設定及使用動態資源分配 (DRA)。
- 如何使用 GKE Inference Gateway 最佳化 LLM 服務效能。
- 如何使用 Agent Sandbox 在 GKE 中安全地執行不受信任的程式碼。
- 如何使用 Google Cloud Managed Service for Prometheus 監控 vLLM 效能。

注意：您必須擁有已連結付款方式的 Google Cloud 帳單帳戶，才能完成這個實驗室。如果您在完成實驗室後立即停用資源，完成這個實驗室的費用大約為 **\$15 美元**。

步驟 2 · 約 0 分鐘

2. 設定和需求

專案設定

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

步驟 3 · 約 15 分鐘

3. 佈建基礎架構和動態資源分配 (DRA)

在第一個步驟中，您將設定 GKE 叢集，使用新式加速器分配 (DRA) 功能，而非舊版裝置外掛程式。這樣一來，您就能彈性分享及分配 GPU 或 TPU，用於程式碼生成工作負載。

必要條件： GKE Standard 叢集必須執行 1.34 以上版本，才能支援 DRA。

啟用 Google Cloud API

啟用本程式碼研究室所需的 Google Cloud API，特別是 Compute Engine 和 Kubernetes Engine API。

```
gcloud services enable compute.googleapis.com container.googleapis.com
networkservices.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com
telemetry.googleapis.com cloudtrace.googleapis.com aiplatform.googleapis.com
```

設定環境變數

為簡化設定流程，請定義環境變數。您可以視需要調整區域或命名慣例。

```
export PROJECT_ID=$(gcloud config get-value project)
export ZONE=us-central1-a
export CLUSTER_NAME=ai-agent-cluster
export NODEPOOL_NAME=dra-accelerator-pool

gcloud config set project $PROJECT_ID
gcloud config set compute/region $ZONE
```

建立工作目錄

為本實驗室建立專屬的工作目錄，並前往該目錄，確保檔案井然有序：

```
mkdir -p ~/gke-ai-agent-lab
cd ~/gke-ai-agent-lab
```

設定權限 (選用)

如果您在受限專案或共用環境中執行作業，請確保您的帳戶具備建立叢集及執行建構作業的必要權限：

```
export MY_ACCOUNT=$(gcloud config get-value account)

# Grant Container Admin to create clusters and manage nodes if needed
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$MY_ACCOUNT" \
  --role="roles/container.admin"

# Grant Cloud Build Builder to run builds
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$MY_ACCOUNT" \
  --role="roles/cloudbuild.builds.builder"
```

建立 GKE 叢集

GKE Standard 叢集必須執行 1.34 以上版本，才能支援 DRA。您也需要啟用 Gateway API 控制器，才能支援智慧型推論排程。

您將為本實驗室建立新的虛擬私有雲網路和子網路。

首先，請建立虛擬私有雲網路：

```
gcloud compute networks create ai-agent-network --subnet-mode=custom
```

接著，為 GKE 節點建立子網路：

```
gcloud compute networks subnets create ai-agent-subnet \
  --network=ai-agent-network \
  --range=10.0.0.0/20 \
  --region=us-central1
```

Gateway API (`gke-17-regional-internal-managed`) 也需要專屬子網路來代管 Envoy Proxy。在新網路中建立這個僅限 Proxy 的子網路：

```
gcloud compute networks subnets create proxy-only-subnet \
  --purpose=REGIONAL_MANAGED_PROXY \
  --role=ACTIVE \
  --region=us-central1 \
  --network=ai-agent-network \
  --range=192.168.10.0/24
```

現在，使用新的網路和子網路建立叢集：

```
gcloud beta container clusters create $CLUSTER_NAME \
  --zone $ZONE \
  --num-nodes 1 \
  --machine-type n2-standard-4 \
  --workload-pool=${PROJECT_ID}.svc.id.goog \
  --gateway-api=standard \
  --managed-otel-scope=COLLECTION_AND_INSTRUMENTATION_COMPONENTS \
  --network=ai-agent-network \
  --subnetwork=ai-agent-subnet
```

建立停用預設外掛程式的節點集區

如要將裝置管理權交給 DRA，您必須建立節點集區，明確停用預設 GPU 驅動程式安裝和標準裝置外掛程式。

執行下列 `gcloud` 指令，使用必要的 DRA 標籤佈建 GPU 節點集區 (例如使用 NVIDIA L4)：

```
gcloud container node-pools create $NODEPOOL_NAME \
  --cluster=$CLUSTER_NAME \
  --location=$ZONE \
  --machine-type=g2-standard-24 \
  --accelerator=type=nvidia-l4,count=2,gpu-driver-version=disabled \
  --node-labels=gke-no-default-nvidia-gpu-device-plugin=true,nvidia.com/gpu.present=true \
  --num-nodes 3
```

如果 GPU 節點集區建立失敗，可能是因為沒有現貨。請稍後再試。您也可以直接跳到「使用 Agent Sandbox 執行安全程式碼」步驟，設定 Agent Sandbox 並部署 Agent。如果自架模型無法使用，代理程式會改用 Vertex AI。您將無法設定 DRA、llm-d 和推論閘道，但仍可探索 Agent 和 Agent Sandbox。

透過 DaemonSet 安裝 NVIDIA 驅動程式

使用預先設定的 Google Cloud DaemonSet，在節點上手動安裝必要的 NVIDIA 裝置基本驅動程式：

```
kubectl apply -f https://raw.githubusercontent.com/GoogleCloudPlatform/container-engine-accelerators/master/nvidia-driver-installer/cos/daemonset-preloaded.yaml
```

安裝 DRA 驅動程式

接著，請在叢集中安裝特定 DRA 驅動程式。如果是 NVIDIA GPU，您可以透過 Helm 部署：


```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update
helm install nvidia-dra-driver-gpu nvidia/nvidia-dra-driver-gpu \
  --version="25.3.2" --create-namespace --namespace=nvidia-dra-driver-gpu \
  --set nvidiaDriverRoot="/home/kubernetes/bin/nvidia/" \
  --set gpuResourcesEnabledOverride=true \
  --set resources.computeDomains.enabled=false \
  --set kubeletPlugin.priorityClassName="" \
  --set 'kubeletPlugin.tolerations[0].key=nvidia.com/gpu' \
  --set 'kubeletPlugin.tolerations[0].operator=Exists' \
  --set 'kubeletPlugin.tolerations[0].effect=NoSchedule'
```

瞭解 DeviceClass

您不必手動編寫或套用 `DeviceClass` YAML。為 DRA 設定 GKE 基礎架構並安裝驅動程式後，節點上執行的 DRA 驅動程式會自動在叢集中建立 `DeviceClass` 物件。

設定 `ResourceClaimTemplate`

如要讓 `llm-d` Pod 動態要求這些加速器，請建立 `ResourceClaimTemplate`。這個範本會定義要求的裝置設定，並告知 Kubernetes 為工作負載自動建立每個 Pod 專屬的 `ResourceClaim`。

執行下列指令來建立 `claim-template.yaml`：

```
cat > claim-template.yaml <<EOF
apiVersion: resource.k8s.io/v1
kind: ResourceClaimTemplate
metadata:
  name: gpu-claim-template
spec:
  spec:
    devices:
      requests:
      - name: single-gpu
        exactly:
          deviceClassName: gpu.nvidia.com
          allocationMode: ExactCount
          count: 1
EOF
```

將範本套用至叢集：

```
kubectl apply -f claim-template.yaml
```

步驟 4 · 約 15 分鐘

4. 使用 llm-d 和 DRA 部署智慧型推論排程

在本步驟中，您會在智慧型 Envoy 負載平衡器後方部署大型語言模型，並透過推論排程器進行強化。這項設定會套用前置字串快取感知路徑，進而最佳化提供模型。GKE Inference Gateway 可辨識微服務之間的共用內容，並將要求智慧轉送至相同的模型副本，盡可能提高快取命中率、縮短第一個詞元生成時間，並提升每美元的效能。

準備環境

設定目標命名空間。

```
export NAMESPACE=ai-agents
kubectl create namespace $NAMESPACE
```

安全地儲存 Hugging Face 權杖，這是提取模型權重的必要條件。

```
# Replace with your actual Hugging Face token
kubectl create secret generic llm-d-hf-token \
  --from-literal=HF_TOKEN="your_hugging_face_token" \
  -n $NAMESPACE
```

建立 Helm 設定檔

模型服務和推論閘道擴充功能的設定，是以官方 [llm-d 指南](#) 為依據。

首先，請為模型服務建立 `ms-values.yaml` 檔案：

```
cat <<EOF > ms-values.yaml
multinode: false

modelArtifacts:
  uri: "hf://Qwen/Qwen2.5-Coder-14B-Instruct"
  name: "Qwen/Qwen2.5-Coder-14B-Instruct"
  size: 50Gi # Slightly larger than the default to accommodate weights
  authSecretName: "llm-d-hf-token"
  labels:
    llm-d.ai/inference-serving: "true"
    llm-d.ai/guide: "inference-scheduling"
    llm-d.ai/accelerator-variant: "gpu"
    llm-d.ai/accelerator-vendor: "nvidia"
    llm-d.ai/model: "qwen-2-5-coder-14b"

routing:
  proxy:
    enabled: false # removes sidecar from deployment - no PD in inference scheduling
    targetPort: 8000 # controls vLLM port to matchup with sidecar if deployed

accelerator:
  dra: true
  type: "nvidia"
  resourceClaimTemplates:
    nvidia:
      class: "gpu.nvidia.com"
      match: "exactly"
      name: "gpu-claim-template"

decode:
  create: true
  tolerations:
    - key: "nvidia.com/gpu"
      operator: "Exists"
      effect: "NoSchedule"
  parallelism:
    tensor: 2
    data: 1
  replicas: 3
  monitoring:
    podmonitor:
      enabled: true
      portName: "vllm"
      path: "/metrics"
      interval: "30s"
  containers:
    - name: "vllm"
      image: ghcr.io/llm-d/llm-d-cuda:v0.5.1
      modelCommand: vllmServe
      args:
        - "--disable-uvicorn-access-log"
        - "--gpu-memory-utilization=0.85"
```

```
- "--enable-auto-tool-choice"
- "--tool-call-parser"
- "hermes"
ports:
  - containerPort: 8000
    name: vllm
    protocol: TCP
resources:
  limits:
    cpu: '16'
    memory: 64Gi
  requests:
    cpu: '16'
    memory: 64Gi
mountModelVolume: true
volumeMounts:
  - name: metrics-volume
    mountPath: /.config
  - name: shm
    mountPath: /dev/shm
  - name: torch-compile-cache
    mountPath: /.cache
startupProbe:
  httpGet:
    path: /v1/models
    port: vllm
  initialDelaySeconds: 15
  periodSeconds: 30
  timeoutSeconds: 5
  failureThreshold: 120
livenessProbe:
  httpGet:
    path: /health
    port: vllm
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /v1/models
    port: vllm
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3
volumes:
  - name: metrics-volume
    emptyDir: {}
  - name: torch-compile-cache
    emptyDir: {}
  - name: shm
    emptyDir:
      medium: Memory
```

```
    sizeLimit: 20Gi
prefill:
  create: false
EOF
```

接著，為 GKE Inference Gateway 擴充功能建立 `gaie-values.yaml` 檔案：

```
cat <<EOF > gaie-values.yaml
inferenceExtension:
  replicas: 1
  image:
    name: llm-d-inference-scheduler
    hub: ghcr.io/llm-d
    tag: v0.6.0
    pullPolicy: Always
  extProcPort: 9002
  pluginsConfigFile: "default-plugins.yaml"
  tracing:
    enabled: false
  monitoring:
    interval: "10s"
    prometheus:
      enabled: true
      auth:
        secretName: inference-scheduling-gateway-sa-metrics-reader-secret
inferencePool:
  targetPorts:
    - number: 8000
  modelServerType: vllm
  modelServers:
    matchLabels:
      llm-d.ai/inference-serving: "true"
      llm-d.ai/guide: "inference-scheduling"
EOF
```

瞭解設定

這項設定會建立高效能的推論堆疊，並提供下列主要功能：

- **模型選擇**：使用 **Qwen 2.5 Coder 14B** 模型 (`modelArtifacts`)，該模型專為程式碼生成和工具使用而設計。
- **DRA 整合**： `accelerator` 區段會啟用動態資源分配 (`dra: true`)，以 `gpu.nvidia.com` 裝置類別和先前建立的 `gpu-claim-template` 為目標。
- **提升效能**：
 - `parallelism.tensor: 2` 會在 GPU 之間設定張量平行處理。
 - `args`，確保程式設計代理能有效使用工具。 `--enable-auto-tool-choice`
 - 縮減的 `cpu` 和 `memory` 要求符合 `g2-standard-24` 機型。
- **智慧型轉送**：設定 Inference Gateway 擴充功能 (`gaie-values.yaml`) 監控 `vllm` 模型伺服器，並轉送要求，盡量提高 KV 快取命中率。

透過 Helm 部署 Inference Scheduling Stack

現在，請新增 llm-d Helm 存放區，並個別部署基礎架構、閘道擴充功能和模型服務。

首先，請新增必要存放區：

```
helm repo add llm-d-infra https://llm-d-incubation.github.io/llm-d-infra/
helm repo add llm-d-modelservice https://llm-d-incubation.github.io/llm-d-modelservice/
helm repo update
```

部署基礎架構先決條件

這個圖表會安裝堆疊所需的基準閘道設定。

```
helm install infra-is llm-d-infra/llm-d-infra \
  --namespace $NAMESPACE \
  --set gateway.gatewayClassName=gke-l7-rilb \
  --set gateway.gatewayParameters.enabled=false \
  --set gateway.gatewayParameters.istio.accessLogging=false
```

部署 GKE Inference Gateway 擴充功能

這個步驟會部署 InferencePool 和 Endpoint Picker，監控模型的 KV 快取，做出智慧型路徑決策。

```
helm install gaie-is oci://registry.k8s.io/gateway-api-inference-
extension/charts/inferencepool \
  --version v1.3.1 \
  --namespace $NAMESPACE \
  -f gaie-values.yaml \
  --set provider.name=gke \
  --set inferenceExtension.monitoring.prometheus.enabled=true
```

部署模型服務

最後，部署 LLM 服務，該服務現在會使用 DRA 安全地聲明 L4 GPU。

```
helm install ms-is llm-d-modelservice/llm-d-modelservice \
  --version v0.4.7 \
  --namespace $NAMESPACE \
  -f ms-values.yaml \
  --set decode.monitoring.podmonitor.enabled=false
```

為 vLLM 啟用 Google Cloud Observability

一般 Helm 資訊圖表通常會嘗試部署標準 Prometheus Operator `PodMonitor` 資源 (monitoring.coreos.com/v1)，如果您未安裝這些 CRD，可能會導致錯誤。

請勿切換 Helm 的內建監控切換鈕，而是將其保持為 `false`，並使用相容的 monitoring.googleapis.com/v1 API 群組手動套用 Google Cloud Managed Prometheus (GMP) `PodMonitoring` 資源。

執行下列指令來建立 `podmonitoring.yaml`：

```
cat > podmonitoring.yaml <<EOF
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: ms-vllm-metrics
spec:
  selector:
    matchLabels:
      llm-d.ai/model: "qwen-2-5-coder-14b" # Matches the label in values.yaml
  endpoints:
    - port: 8000 # vllm port
      interval: 30s
      path: /metrics
EOF
```

將 `PodMonitoring` 資源套用至叢集：

```
kubectl apply -f podmonitoring.yaml -n $NAMESPACE
```

驗證安裝

確認元件已成功安裝。您應該會看到命名空間中所有三個 Helm 版本都處於啟用狀態，且對應的 Pod 正在初始化。

```
helm ls -n $NAMESPACE
kubectl get pods -n $NAMESPACE
```

ms-is Pod 大約需要 5 到 10 分鐘才能啟動。輸出內容應該會類似下列示例：

NAME	NAMESPACE	REVISION	UPDATED
STATUS	CHART		APP VERSION
gaie-is	ai-agents	1	2026-03-28 16:51:41.055881618 +0000 UTC
deployed	inferencepool-v1.3.1		v1.3.1
infra-is	ai-agents	1	2026-03-28 16:51:03.71042542 +0000 UTC
deployed	llm-d-infra-v1.4.0		v0.4.0
ms-is	ai-agents	1	2026-03-28 17:30:00.341918958 +0000 UTC
deployed	llm-d-modelservice-v0.4.7		v0.4.0

NAME	READY	STATUS	RESTARTS	AGE
gaie-is-epp-848965cb4-78ktp	1/1	Running	0	10m
ms-is-llm-d-modelservice-decode-67548d5f8c-f25f4	1/1	Running	0	6m2s
ms-is-llm-d-modelservice-decode-67548d5f8c-rblvs	1/1	Running	0	6m2s
ms-is-llm-d-modelservice-decode-67548d5f8c-w6fcd	1/1	Running	0	6m2s

檢查 Google Cloud 控制台時，您的 `ms-is-llm-d-modelservice-decode` 部署作業可能會暫時顯示紅色警告：「Does not have minimum availability」，且副本會列為 3/3。這是正常的狀況。Pod 正在等待從 HuggingFace 下載大型 Qwen 模型權重 (約 50 GiB)。請稍候幾分鐘再返回，權重快取完成後，就會變成綠色。

5. 使用 GKE Inference Gateway 設定智慧型路徑

在步驟 4 中，部署 `llm-d` Helm 圖表會自動佈建 Gateway 和 `InferencePool` 物件。`InferencePool` 會將共用相同基礎模型和運算設定的 `vllm` 提供模型 Pod 分組。

現在需要設定 `InferenceObjective`，為程式碼編寫代理的要求設定優先順序，並設定 `HTTPRoute`，指示 Gateway 如何傳送傳入流量，同時運用 Endpoint Picker 盡量提高 KV 快取命中率。

驗證自動生成的資源

首先，請確認 `llm-d` Helm 資訊圖表已成功建立 Gateway 和 `InferencePool` 資源。

```
kubectl get gateway,inferencepool -n $NAMESPACE
```

您應該會看到名為 `infra-is-inference-gateway` 的閘道，以及名為 `gaie-is` 的 `InferencePool`。類似於：

NAME				CLASS
ADDRESS	PROGRAMMED	AGE		
gateway.gateway.networking.k8s.io/infra-is-inference-gateway				gke-l7-regional-internal-
managed	10.128.0.5	True	13m	

NAME	AGE
inferencepool.inference.networking.k8s.io/gaie-is	12m

建立 HTTPRoute

`HTTPRoute` 資源會將閘道對應至後端 `InferencePool`。這會告知 GKE Inference Gateway 分析傳入的要求主體，並根據共用內容動態轉送要求，盡可能提高前置字元快取命中率。

執行下列指令來建立 `httproute.yaml`：


```
cat > httproute.yaml <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: agent-route
spec:
  parentRefs:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: infra-is-inference-gateway
  rules:
  - backendRefs:
    - group: inference.networking.k8s.io
      kind: InferencePool
      name: gaie-is
      weight: 1
    matches:
    - path:
        type: PathPrefix
        value: /
EOF
```

將路徑套用至叢集：

```
kubectl apply -f httproute.yaml -n $NAMESPACE
```

6. 使用 Agent Sandbox 安全執行程式碼

高效能的推論後端已開始運作，接下來請準備安全環境，讓 AI 生成的程式碼實際執行，並使用 Agent Sandbox 與叢集安全隔離。

部署 Agent Sandbox 控制器

當 AI 代理程式產生及執行程式碼時，基本上是在您的基礎架構上執行不受信任的工作負載。如果代理程式產生惡意程式碼，可能會嘗試掃描內部網路或利用基礎主機節點。

GKE Agent Sandbox 採用 **gVisor**，這是一種開放原始碼容器執行階段，可為每個容器提供專屬的客體核心。這樣可防止不受信任的程式碼直接呼叫主機節點的系統。

套用正式發布資訊清單，部署 Agent Sandbox 控制器及其必要元件：

```
kubectl apply \
  -f https://github.com/kubernetes-sigs/agent-sandbox/releases/download/v0.1.0/manifest.yaml \
  -f https://github.com/kubernetes-sigs/agent-sandbox/releases/download/v0.1.0/extensions.yaml
```

設定沙箱範本和暖池

接著，我們建立 `SandboxTemplate`，做為 Python 分析環境的可重複使用藍圖，並明確指定 `gvisor` 執行階段類別。如要簡化部署作業，不必在標準叢集上管理手動節點集區，我們可以運用任何標準 `autopilot`

ComputeClass 可動態佈建代管運算節點，原生支援 gVisor 工作負載！

由於初始化安全核心可能會增加延遲時間，因此我們也會部署 `SandboxWarmPool`。這項設定可確保系統會預先初始化指定數量的沙箱，讓程式碼生成代理程式能認領沙箱，並在不到一秒的時間內開始執行程式碼。

首先，為代理程式沙箱執行階段建立新的命名空間：

```
kubectl create namespace agent-sandbox
```

將下列內容儲存為 `sandbox-template-and-pool.yaml`：

```

cat > sandbox-template-and-pool.yaml <<EOF
apiVersion: extensions.agents.x-k8s.io/v1alpha1
kind: SandboxTemplate
metadata:
  name: python-runtime-template
  namespace: agent-sandbox
spec:
  podTemplate:
    metadata:
      labels:
        sandbox: python-sandbox-example
    spec:
      runtimeClassName: gvisor
      nodeSelector:
        cloud.google.com/compute-class: autopilot
      containers:
      - name: python-runtime
        image: registry.k8s.io/agent-sandbox/python-runtime-sandbox:v0.1.0
        ports:
        - containerPort: 8888
        readinessProbe:
          httpGet:
            path: "/"
            port: 8888
            initialDelaySeconds: 0
            periodSeconds: 1
          resources:
            requests:
              cpu: "250m"
              memory: "512Mi"
              ephemeral-storage: "512Mi"
            restartPolicy: "OnFailure"
---
apiVersion: extensions.agents.x-k8s.io/v1alpha1
kind: SandboxWarmPool
metadata:
  name: python-sandbox-warmpool
  namespace: agent-sandbox
spec:
  replicas: 2
  sandboxTemplateRef:
    name: python-runtime-template
EOF

```

套用設定：

```
kubectl apply -f sandbox-template-and-pool.yaml
```

請等待最多 2 到 3 分鐘，讓暖池 Pod 初始化。您可以使用下列指令，確認這些節點是否已從 **Pending** (基礎運算資源擴充時) 成功轉換為 **Running**：

```
kubectl get pods -n agent-sandbox -w
```

Pod 執行後，按下 `Ctrl+C` 鍵即可退出指令。

看到兩個 `python-sandbox-warmup-***` Pod 列為「Ready」`Running` 和「Ready」`1/1` 後，安全執行環境就會預先暖機，隨時可供使用！

部署沙箱路由器

我們的程式碼生成代理會透過 Sandbox Router，將執行指令安全地派送至獨立 Pod。

執行下列指令來建立 `sandbox-router.yaml`：

```

cat > sandbox-router.yaml <<EOF
apiVersion: v1
kind: Service
metadata:
  name: sandbox-router-svc
  namespace: agent-sandbox
spec:
  type: ClusterIP
  selector:
    app: sandbox-router
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sandbox-router-deployment
  namespace: agent-sandbox
spec:
  replicas: 2
  selector:
    matchLabels:
      app: sandbox-router
  template:
    metadata:
      labels:
        app: sandbox-router
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: sandbox-router
      containers:
        - name: router
          image: us-centrall-docker.pkg.dev/k8s-staging-images/agent-sandbox/sandbox-
router:v20260225-v0.1.1.post3-10-ga5bcb57
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet:

```

```

    path: /healthz
    port: 8080
    initialDelaySeconds: 10
    periodSeconds: 10
  resources:
    requests:
      cpu: "250m"
      memory: "512Mi"
    limits:
      cpu: "1000m"
      memory: "1Gi"
  securityContext:
    runAsUser: 1000
    runAsGroup: 1000
EOF

```

套用設定：

```
kubectl apply -f sandbox-router.yaml
```

實作網路隔離

如要進一步鎖定執行環境，防止任何未經授權的橫向移動，請套用網路政策。這會「隔離」沙箱，使其無法連線至 Google Cloud 中繼資料伺服器或其他機密內部網路。

將下列內容儲存為 `sandbox-policy.yaml`：

```

cat > sandbox-policy.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-sandbox-egress
  namespace: agent-sandbox
spec:
  podSelector:
    matchLabels:
      sandbox: python-sandbox-example
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0
        except:
        - 169.254.169.254/32 # Block metadata server
EOF

```

套用政策：

```
kubectl apply -f sandbox-policy.yaml
```

驗證元件

如要確保已完整設定獨立程式碼沙箱叢集層，請執行下列狀態驗證指令：

首先，請確認沙箱 Pod 和路由器正在執行且已準備就緒

```
kubectl get pods -n agent-sandbox
```

輸出內容應如下所示：

NAME	READY	STATUS	RESTARTS	AGE
python-sandbox-warmpool-7zlkv	1/1	Running	0	3m25s
python-sandbox-warmpool-cxln2	1/1	Running	0	3m25s
sandbox-router-deployment-668dfbbbb6-g9mpd	1/1	Running	0	42s
sandbox-router-deployment-668dfbbbb6-ppllz	1/1	Running	0	42s

驗證 Sandbox Router 負載平衡器 / IP 曝光

```
kubectl get service sandbox-router-svc -n agent-sandbox
```

輸出內容應如下所示：

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
sandbox-router-svc	ClusterIP	34.118.237.244	<none>	8080/TCP	114s

確認輸出網路政策規則是否存在

```
kubectl get networkpolicy restrict-sandbox-egress -n agent-sandbox
```

輸出內容應如下所示：

NAME	POD-SELECTOR	AGE
restrict-sandbox-egress	sandbox=python-sandbox-example	113s

請確認下列項目：

- python-sandbox-warmpool-*** Pod 處於 Running 和 1/1 狀態。
- sandbox-router-deployment-*** 副本處於 Running 和 1/1 「就緒」狀態。
- sandbox-router-svc 可存取，且 restrict-sandbox-egress 政策已成功保護任何相符的沙箱標籤。

安全執行環境已受到保護並完成初始化，現在可以部署實際的作業大腦：程式碼生成代理程式！

7. 建構及部署程式碼生成代理 (ADK)

設定好安全執行沙箱和高效能 LLM 後端後，我們現在可以建構系統的「大腦」：使用 **Agent Development Kit (ADK)** 的程式碼生成代理。

這個代理程式的設計目標是成為專業的 Python 開發人員，與只能生成文字的一般聊天機器人不同，這個代理配備**程式碼執行工具**，可互動式解決問題。這項功能會依序執行下列步驟：

1. 根據要求編寫 Python 程式碼。
2. 在步驟 6 中設定的 GKE 代理程式沙箱中，安全地執行程式碼。
3. 驗證輸出內容，或讀取執行期間發生的任何錯誤。
4. 交付經過測試且可正常運作的解決方案。

我們讓代理程式存取安全沙箱執行環境，以便驗證自身邏輯並自動偵錯失敗，大幅提升軟體開發能力！

開發 ADK Reasoning Agent

首先，我們編寫 Python 邏輯，定義代理程式的行為，並為其配備在步驟 6 中建立的 Sandbox 工具。在本節中，我們也會設定混合模型策略：代理程式會優先使用在 GKE 叢集上執行的自架 Qwen 模型，但如果本機模型速度緩慢或無法使用，則會自動改用 Vertex AI 上的 Gemini 2.5 Flash，確保高可靠性。

為代理程式碼建立新目錄：

```
mkdir -p ~/gke-ai-agent-lab/root_agent
cd ~/gke-ai-agent-lab
```

建立名為 `root_agent/agent.py` 的檔案，並加入以下內容：


```

cat <<'EOF' > root_agent/agent.py
import os
from google.adk.agents import Agent
from google.adk.models.lite_llm import LiteLlm
from k8s_agent_sandbox import SandboxClient
import requests

# Instantiate the client globally to track sandboxes
sandbox_client = SandboxClient()

def run_python_code(code: str) -> str:
    """
    Executes Python code safely in the GKE Agent Sandbox.
    Use this tool whenever you need to execute code to solve a problem.
    """
    sandbox = sandbox_client.create_sandbox(template="python-runtime-template",
    namespace="agent-sandbox")
    try:
        command = f"python3 -c \"{code}\""
        result = sandbox.commands.run(command)
        if result.stderr:
            return f"Error: {result.stderr}"
        return result.stdout
    finally:
        # Ensure the sandbox is deleted after use to avoid leaking resources
        sandbox_client.delete_sandbox(sandbox.claim_name, namespace="agent-sandbox")

# Define the ADK Agent with a fallback mechanism.
# It prioritizes the Qwen 2.5 Coder model running on our Inference Gateway.
# If the local model is unavailable, it falls back to Gemini 2.5 Flash on Vertex AI.
root_agent = Agent(
    name="CodeGenerationAgent",
    model=LiteLlm(
        model="openai/Qwen/Qwen2.5-Coder-14B-Instruct",
        fallbacks=["vertex_ai/gemini-2.5-flash"],
        timeout=10
    ),
    instruction="""
You are an expert Python developer.
1. Write Python code to solve the user's problem.
2. Execute the code using the `run_python_code` tool to verify it works.
3. Return the exact output and a brief explanation of the code.
""",
    tools=[run_python_code]
)
EOF

```

建立 `__init__.py` 檔案，讓 ADK 辨識模組：

```
echo "from . import agent" > ~/gke-ai-agent-lab/root_agent/__init__.py
```

設定環境變數。ADK 應用程式需要閘道的 IP 位址，才能順利傳送 LLM 要求。由於 ADK 支援標準的 Open-AI 相容端點 (vLLM 會透過 Gateway 提供)，因此我們可以覆寫預設的 API 基礎網址！

```
export GATEWAY_IP=$(kubectl get gateway infra-is-inference-gateway -n $NAMESPACE -o
jsonpath='{.status.addresses[0].value}')

cat <<EOF > ~/gke-ai-agent-lab/root_agent/.env
OPENAI_API_BASE=http://${GATEWAY_IP}/v1
OPENAI_API_KEY=no-key-required
# Vertex AI settings for fallback (Authentication is handled by Workload Identity)
VERTEXAI_PROJECT=$PROJECT_ID
VERTEXAI_LOCATION=${ZONE%-[a-z]}
EOF
```

將代理程式應用程式容器化

我們需要封裝代理程式，才能在 GKE 中安全地執行。

在 `~/gke-ai-agent-lab` 中建立 `Dockerfile`，安裝 `kubectl`、ADK 程式庫和 Agent Sandbox 用戶端：

```
cat <<'EOF' > ~/gke-ai-agent-lab/Dockerfile
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y git curl \
    && curl -LO "https://dl.k8s.io/release/${curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" \
    && install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl \
    && rm kubectl && apt-get clean && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir "google-adk[extensions,otel-gcp]>=1.27.4" litellm pandas
"git+https://github.com/kubernetes-sigs/agent-
sandbox.git@main#subdirectory=clients/python/agent-sandbox-client" \
    "opentelemetry-instrumentation-google-genai>=0.4b0" \
    "opentelemetry-exporter-otlp" \
    "opentelemetry-instrumentation-vertexai>=2.0b0"

COPY ./root_agent /app/root_agent

EXPOSE 8080

ENTRYPOINT ["adk", "web", "--host", "0.0.0.0", "--port", "8080", "--otel_to_cloud"]
EOF
```

建立 Artifact Registry 存放區，用於儲存容器映像檔。

```
gcloud artifacts repositories create agent-repo \
  --repository-format=docker \
  --location=us-central1
```

使用 Cloud Build 建構及推送容器映像檔。

```
gcloud builds submit --tag us-central1-docker.pkg.dev/$PROJECT_ID/agent-repo/code-agent:v1
~/gke-ai-agent-lab/
```

如果這個指令無法正常運作，請嘗試使用下列指令重新匯出 PROJECT_ID、ZONE 和 GATEWAY_IP 環境變數：

```
export PROJECT_ID=$(gcloud config get-value project)
export ZONE=$(gcloud config get-value compute/zone)
export GATEWAY_IP=$(kubectl get gateway infra-is-inference-gateway -n $NAMESPACE -o
jsonpath='{.status.addresses[0].value}')
```

使用 RBAC 部署至 GKE

最後，將代理程式部署至叢集。部署作業包含 `Role` 和 `RoleBinding`，可授予代理程式從 `SandboxWarmPool` 聲明執行個體的權限。

這項部署作業會使用 Kubernetes ServiceAccount，讓代理程式與 Sandbox 聲明 API 通訊。由於存取的是本機叢集資源和本機 vLLM 閘道端點，因此不需要 Google IAM ServiceAccount。

為什麼要在 gVisor 中採用標準部署？

在步驟 6 中，我們使用 `SandboxTemplate` 和 `SandboxClaim` API，為生成的 **Python** 程式碼 (工具執行) 建立暫時性、可拋棄的沙箱。

就代理程式網頁版 UI (即 **Brain**) 而言，我們使用標準 Kubernetes `Deployment` 規格搭配 `runtimeClassName: gvisor`。

- **區別：**標準 `SandboxClaims` 是暫時性的，且為零到一 (適合用於不受信任的指令碼)。標準 `Deployment` 持續運作時間長，非常適合需要穩定 Kubernetes `Service` 和負載平衡器的網頁 UI！直接在標準 `Deployment` 上使用 `runtimeClassName: gvisor`，即可在保留標準 `Deployment` 功能的同時，享有 gVisor 核心的隔離功能。

將下列內容儲存為 `deployment.yaml`：

```

cat <<EOF > ~/gke-ai-agent-lab/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: code-agent
  labels:
    app: code-agent
spec:
  replicas: 1
  selector:
    matchLabels:
      app: code-agent
  template:
    metadata:
      labels:
        app: code-agent
    spec:
      serviceAccount: adk-agent-sa
      runtimeClassName: gvisor
      nodeSelector:
        cloud.google.com/compute-class: autopilot
      containers:
        - name: code-agent
          image: us-centrall-docker.pkg.dev/YOUR_PROJECT_ID/agent-repo/code-agent:v1
          imagePullPolicy: Always
          env:
            - name: OPENAI_API_KEY
              value: "no-key-required"
            - name: OPENAI_API_BASE
              value: "http://YOUR_GATEWAY_IP/v1"
            - name: VERTEXAI_PROJECT
              value: "YOUR_PROJECT_ID"
            - name: VERTEXAI_LOCATION
              value: "YOUR_REGION"
            - name: OTEL_SERVICE_NAME
              value: "code-agent"
            - name: GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY
              value: "true"
            - name: OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT
              value: "true"
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: code-agent-service
spec:
  selector:
    app: code-agent
  ports:
    - protocol: TCP

```

```

    port: 80
    targetPort: 8080
    type: LoadBalancer
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: agent-sandbox
  name: sandbox-creator-role
rules:
- apiGroups: [""]
  resources: ["services", "pods", "pods/portforward"]
  verbs: ["get", "list", "watch", "create"]
- apiGroups: ["extensions.agents.x-k8s.io"]
  resources: ["sandboxclaims"]
  verbs: ["create", "get", "list", "watch", "delete"]
- apiGroups: ["agents.x-k8s.io"]
  resources: ["sandboxes"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: adk-agent-binding
  namespace: agent-sandbox
subjects:
- kind: ServiceAccount
  name: adk-agent-sa
  namespace: default
roleRef:
  kind: Role
  name: sandbox-creator-role
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: adk-agent-sa
EOF

```

授許可觀測性的 IAM 權限

如要讓代理程式將遙測資料 (記錄和追蹤) 傳送至 Google Cloud，您必須使用 Workload Identity，將必要權限授予 Kubernetes 服務帳戶 `adk-agent-sa`。

在 Cloud Shell 中執行下列指令：

```
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")

# Grant permission to write logs
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --role=roles/logging.logWriter \
  --
member=principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/${PROJECT_ID}.svc.id.goog/subject/ns/default/sa/adk-agent-sa

# Grant permission to write traces
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --role=roles/cloudtrace.agent \
  --
member=principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/${PROJECT_ID}.svc.id.goog/subject/ns/default/sa/adk-agent-sa

# Grant permission to use Vertex AI
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --role=roles/aiplatform.user \
  --
member=principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/${PROJECT_ID}.svc.id.goog/subject/ns/default/sa/adk-agent-sa
```

執行下列指令，自動將 `YOUR_PROJECT_ID` 替換為實際專案 ID，並套用設定！

```
sed -i "s/YOUR_PROJECT_ID/$PROJECT_ID/g" ~/gke-ai-agent-lab/deployment.yaml
sed -i "s/YOUR_GATEWAY_IP/$GATEWAY_IP/g" ~/gke-ai-agent-lab/deployment.yaml
sed -i "s/YOUR_REGION/$ZONE/g" ~/gke-ai-agent-lab/deployment.yaml
kubectl apply -f ~/gke-ai-agent-lab/deployment.yaml
```

8. 觀察及驗證

現在可以測試完全整合的系統了。

在 UI 中測試程式碼生成代理

找出 ADK 網頁介面的外部 IP：

```
kubectl get services code-agent-service
```

輸出內容應如下所示：

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
code-agent-service	LoadBalancer	34.118.230.182	34.31.250.60	80:32471/TCP	2m14s

1. 開啟瀏覽器並前往 `http://[EXTERNAL-IP]`。
2. 在 ADK 網頁介面中，請務必從右上角的下拉式選單選取「root_agent」。然後提示代理：

```
Write a python script that prints 'Hello from the isolated sandbox'.
```

觀看 Sandbox Claims 直播！當專員處理您的要求時，您會看到動態沙箱建立和刪除的過程。在 Cloud Shell 中開啟新的終端機，然後執行：`kubectl get sandboxclaims -n agent-sandbox --watch`。代理執行程式碼時，您會看到聲明，但聲明會在代理傳回結果後不久消失！

若要觀察代理程式如何運用推論後端和沙箱，請前往下方的「[透過 Cloud Observability 探索模型統計資料](#)」和「[透過 GKE UI 探索代理程式可觀測性](#)」部分，查看資訊主頁。

資料可能需要一到兩分鐘才會顯示在資訊主頁中。請代理程式建立幾個「hello world」的變體，這樣您就有更多資料可供查看。

透過 GKE 使用者介面探索代理程式觀測功能

現在您已執行一些提示，接著來看看遙測資料。這有助於瞭解 Inference Scheduler 和 vLLM 的成效。

存取服務專員資訊主頁

1. 前往「Kubernetes Engine」>「工作負載」頁面。
2. 按一下「code-agent」部署作業，開啟「Deployment Details」(部署作業詳細資料) 頁面。
3. 按一下「可觀測性」分頁標籤。
4. 在可觀測性資訊主頁的左側導覽面板中，您會看到新的「代理程式」專區和子分頁。

探索內容

請瀏覽下列子標籤，查看代理程式應用程式的行為：

- **總覽**：查看工作階段、平均回合數和叫用次數的評量表。
- **模型**：查看代理使用的模型，以及模型呼叫次數、錯誤率和延遲時間。
- **工具**：監控工具呼叫和執行時間長度，瞭解代理程式使用沙箱執行工具的效率。
- **用量**：追蹤權杖用量和標準容器資源分配 (CPU 和記憶體)。
- **代理程式追蹤記錄**：切換至這個分頁，即可查看執行工作階段或原始追蹤記錄範圍的清單。點選任一資料列可開啟飛出視窗，查看所選追蹤記錄的詳細資料！

結合 vLLM 的模型層級指標和 ADK 的應用程式層級遙測資料，您現在可以全面觀察 GKE 上的生成式 AI 代理程式！

透過 Cloud Observability 探索 vLLM 模型統計資料

現在您已執行一些提示，接著來看看遙測資料。這有助於瞭解 Inference Scheduler 和 vLLM 的成效。

存取資訊主頁

1. 前往 **Google Cloud 控制台**。
2. 依序前往「監控」>「資訊主頁」。
3. 搜尋並選取「vLLM Prometheus Overview」資訊主頁。

您也可能會看到 **llm-d** 總覽資訊主頁。在本實驗室中，我們執行的是「統一」部署作業，單一節點會同時處理提示評估 (預先填入) 和權杖生成 (解碼)。當您擴充至**分離式服務** (將預先填入和解碼運算集區分開) 時，**llm-d** 資訊主頁就會派上用場。如要進一步瞭解該拓撲，請參閱 [Google Cloud llm-d Exporter 說明文件](#)。如要深入瞭解標準 vLLM 公開的指標，以及 Google Cloud Managed Prometheus 如何直接擷取這些指標，請參閱 [Google Cloud vLLM Exporter 說明文件](#)。

值得觀察的指標

查看資訊主頁時，請注意下列重要指標，瞭解 GKE 推論閘道和前置字元快取造成的影響：

- **KV 快取利用率** (`vllm:gpu_cache_usage`)：
 - **重要性**：這項指標會顯示用於快取內容的 GPU 記憶體容量。如果這個值很高，表示系統會保留內容，以加快日後的要求。如果多次執行相同提示，您應該會看到這項使用率上升，然後趨於穩定。
- **執行中要求與等待中要求** (`vllm:num_requests_running` 與 `vllm:num_requests_waiting`)：
 - **重要性**：這項指標代表負荷。如果等待中的要求數量偏高，表示節點負載過重。
- **符記處理量** (`vllm:request_prompt_tokens_tot` 和 `vllm:request_generation_tokens_tot`)：
 - **重要性**：追蹤叢集處理的輸入和輸出權杖數量。
- **首次權杖時間 (TTFT)**：
 - **重要性**：這是互動式代理程式的重要指標。搭配使用 GKE Inference Gateway 與前置字元快取感知轉送功能，即可將共用脈絡 (例如系統提示或大型脈絡視窗) 的要求轉送至相同副本，並重複使用現有快取命中，盡量縮短 TTFT！

可嘗試的實驗

請嘗試以下情境，即時查看指標變化，並驗證排程是否正確！

實驗 1：「重複速度」(前置字元快取命中)

1. 向代理程式傳送複雜的提示 (例如「編寫 Python 指令碼來剖析 100 MB 的 CSV 檔案，並計算統計資料。」)。
2. 收到回覆後，請立即再次傳送**完全相同的提示**。
3. 觀察「字首快取命中率」和「第一個權杖時間 (TTFT)」。
 - 您應該會看到：前置字串快取命中率應會升至 100%，而 **TTFT** 應會大幅下降！
 - 意義：GKE Inference Gateway 辨識出共用內容，並將其傳送至完全相同的副本，該副本重複使用了評估的內容快取！

實驗 2：回復至雲端 (模型可靠性)

1. 如要模擬本機 Qwen 模型發生故障，您可以停止推論服務，或在部署中提供虛假的 `OPENAI_API_BASE`。
2. 將 `deployment.yaml` 中的 `OPENAI_API_BASE` 更新為不存在的 IP 或通訊埠，然後套用變更：

```
sed -i "s|value: \"http://$GATEWAY_IP/v1\"|value: \"http://10.0.0.1:8080/v1\"|g" ~/gke-ai-agent-lab/deployment.yaml
kubectl apply -f ~/gke-ai-agent-lab/deployment.yaml
```

3. 等待 Pod 重新啟動，然後在 UI 中向代理程式傳送提示。
 - 您應該會看到：代理程式仍會成功回應！
 - 意義：由於 `fallbacks` 設定，ADK 偵測到本機 Qwen 端點發生故障，並將要求順暢地轉送至 Vertex AI 上的 Gemini 2.5 Flash。請注意，這些對 Vertex AI 的備援呼叫會略過本機 vLLM 推論閘道，因此不會顯示在「Agent Observability」>「Models」資訊主頁中，因為該資訊主頁只會追蹤透過 vLLM 的流量。

瞭解動態資源分配 (DRA) 的強大功能

vLLM 和 Inference Gateway 會盡量以最佳方式傳送及處理要求，但**動態資源分配 (DRA)** 才是讓工作負載能使用正確硬體的關鍵。

DRA 可讓您使用 `ResourceClaimTemplate` 和 `DeviceClasses` 定義彈性硬體資源，進而精細管理叢集中的硬體。

DRA 為何能徹底改變 AI 工作負載：

1. **精細的硬體要求**：使用 DRA 時，您不僅能確保工作負載排在具備合適加速器的機器上執行，還可對這些資源提出要求，確保資源專供與 `ResourceClaim` 相關聯的工作負載使用。
2. **生命週期分離**：裝置聲明與 Pod 生命週期無關，如果 Pod 發生當機情形，GPU 聲明可能會持續存在，因此整體部署作業或其他工作負載物件可以重新啟動，不必等待 GPU 釋出並重新取得。
3. **多供應商標準化**：DRA 為 NVIDIA GPU 和 Google TPU 提供統一的 Kubernetes API。無論是為其中一個平台部署，您都會使用完全相同的結構定義，因此工作負載 YAML 資訊清單非常容易移植！

在本程式碼研究室中，您設定 Helm 值以無縫繫結至 `gpu-claim-template` 時，會看到這項功能實際運作，不會有懸而未決的裝置外掛程式設定阻礙推出作業。

瞭解 llm-d 的角色

vLLM 會評估神經網路權重，**GKE Gateway** 則會將查詢路徑導向 **llm-d**，後者會做為設定層，並將所有項目繫結在一起。

如果沒有 `llm-d`，您就必須從頭編寫原始 Kubernetes 資訊清單，才能宣告 `vLLM` 部署作業、服務連接埠、磁碟區掛接，以及 DRA 資源聲明。

為什麼要在部署作業中使用 llm-d ？

1. **統一設定 (單行覆寫)：** llm-d Helm 資訊套件會將複雜的低階 Kubernetes 資源，整合成簡潔的高階切換選項 (例如設定 `accelerator.dra: true`)。
2. **預先審查的「照明路徑」：** llm-d 存放區包含專家已進行基準測試和測試的設定。部署 llm-d-modelservice 時，您會收到 GPU 記憶體使用率的最佳化預設值、建議的探測時間 (liveness/readiness)，以及指標擷取的正確曝光次數。
3. **無縫可觀測性對應：** llm-d 可確保標準容器通訊埠和擷取路徑 (`/metrics`) 正確公開，讓您輕鬆將部署作業連線至 Google Cloud Monitoring，不必手動偵錯。

簡而言之，llm-d 提供可重複使用的架構藍圖，因此開發人員不必每次在 GKE 上部署推論堆疊時，都從頭開始設計。

深入探討：GKE Inference Gateway

標準第 7 層負載平衡器會查看路徑 (`/v1/completions`) 或 Cookie 等 HTTP 標頭，**GKE Inference Gateway** 則更深入，專為生成式 AI 流量設計。

如何提升效能和效率：

1. **內容感知路徑 (提示雜湊)：** GKE 推論閘道會攔截 JSON 要求主體。這項服務會計算提示的雜湊值，並追蹤哪些後端副本已將這些權杖保留在 GPU 記憶體 (KV 快取) 中。
2. **盡量提高快取命中率：** 在測試中，當您重複提示時，Gateway 會將提示傳送至完全相同的副本。評估提示需要大量運算資源，重複使用快取可避免「重新讀取」提示，節省費用和 GPU 時間。
3. **大幅縮短首次權杖時間 (TTFT)：** TTFT 是面向人類的代理程式的重要可用性指標。命中快取後，模型就能在幾毫秒內開始生成權杖，而非幾秒。
4. **智慧型負載分配：** 如果某個副本的 VRAM 完全充滿快取命中，閘道可以動態將新的提示路由至有空閒的其他副本，在效率與可用性之間取得平衡。

Agent Sandbox 如何降低風險

在本實驗室中，我們示範了 **Agent Sandbox** 如何提供兩層隔離機制，保護基礎架構免於 AI 代理相關風險：

- **隔離執行工具：** 代理會在暫時性沙箱中執行生成的程式碼。這可確保 LLM 產生的不受信任程式碼會在安全的隔離環境中執行，保護代理和叢集。
- **快速啟動：** 使用 WarmPool 時，新的沙箱會在不到一秒內啟動，準備好執行程式碼。
- **隔離代理程式本身：** 我們也在啟用 gVisor 的節點中 (透過 `runtimeClassName: gvisor`) 執行代理程式應用程式本身，針對代理程式依附元件中的供應鏈安全漏洞，提供深層防禦措施。

以下說明為何這能建立如此堅實的安全邊界：

1. **系統呼叫攔截：** gVisor 會在系統呼叫送達主機 Linux 核心前攔截。這項設定會封鎖試圖脫離容器，存取主機節點的攻擊。
2. **限制橫向移動：** 搭配網路政策使用，即使環境遭到入侵，也無法掃描內部中繼資料伺服器，或轉向叢集中的其他敏感服務。

在沙箱中執行完整代理程式

在本實驗室中，我們使用沙箱做為持續性代理程式應用程式的工具。不過，為了確保最高安全性 (特別是在處理私密資料或為多位不受信任的使用者提供服務時)，您可以為每個工作階段或使用者的，在專屬沙箱中執行整個代理程式應用程式。確保代理程式的記憶體、狀態和執行環境完全隔離，並在工作階段完成後立即銷毀。

9. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟刪除這些資源。

即使刪除專案，如果資源仍處於啟用狀態，在 30 天的復原期間內仍可能會產生費用。為避免發生這種情況，請先刪除個別資源，再刪除專案。

刪除個別資源

1. 刪除 GKE 叢集：

```
gcloud container clusters delete $CLUSTER_NAME --zone $ZONE --quiet
```

2. 刪除 Artifact Registry 存放區：

```
gcloud artifacts repositories delete agent-repo --location=us-central1 --quiet
```

3. 刪除虛擬私有雲網路：

```
gcloud compute networks delete ai-agent-network --quiet
```

刪除專案

如果不再需要專案，可以先移除資源，然後刪除專案：

```
gcloud projects delete $PROJECT_ID
```

10. 摘要

恭喜！您已成功在 GKE 上建構及部署安全的高效能程式碼生成代理程式。

您學到的內容

- 瞭解如何在 GKE 中設定及使用**動態資源分配 (DRA)** 管理 GPU 資源。
- 如何使用 **GKE Inference Gateway**，透過前置字元快取感知路由最佳化 LLM 服務效能。
- 如何使用 **Agent Sandbox (gVisor)** 在 GKE 上安全地執行不受信任的程式碼。
- 如何使用 **Google Cloud Managed Service for Prometheus** 監控 vLLM 效能。
- 瞭解如何使用 ADK 和 GKE Managed OpenTelemetry，設定及查看代理程式觀測功能。

後續步驟與參考資料

- 代理程式沙箱：瞭解 [GKE Sandbox](#)和 [GKE 沙箱 Pod](#)。
 - llm-d：請參閱 [llm-d 指南](#)，並查看 [llm-d GitHub 存放區](#)。
 - 動態資源分配：瞭解 [GKE 的 DRA](#)。
 - **GKE Inference Gateway**：瞭解 [Inference Gateway 概念](#)。
 - 更多程式碼研究室：如需更多教學課程，請前往 [Google Cloud 程式碼研究室](#)。
-